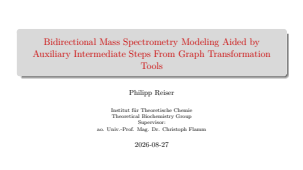
Bidirectional Mass Spectrometry Modeling Aided by Auxiliary Intermediate Steps From Graph Transformation Tools

Philipp Reiser

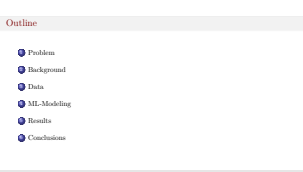
Institut für Theoretische Chemie
Theoretical Biochemistry Group
Supervisor:
ao. Univ.-Prof. Mag. Dr. Christoph Flamm

2026-08-27



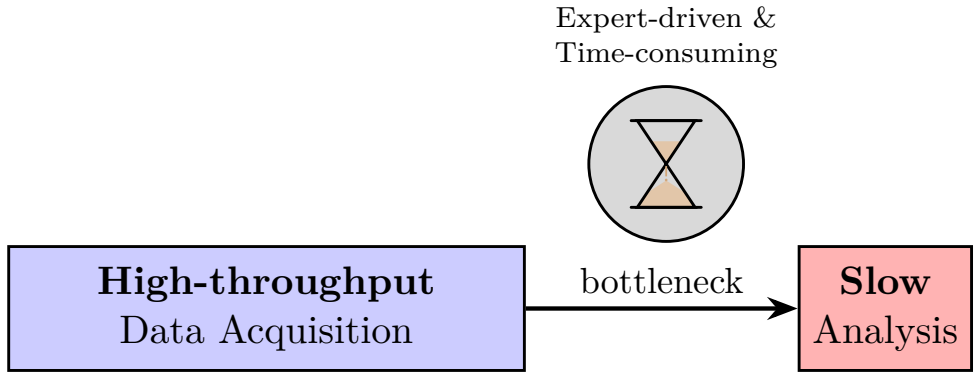
Welcome, everyone, and thank you for coming — a particular welcome to the chair and to the members of the committee. My name is Philipp Reiser, and I will be presenting my master's thesis, carried out in the Theoretical Biochemistry Group under the supervision of Christoph Flamm.

Outline

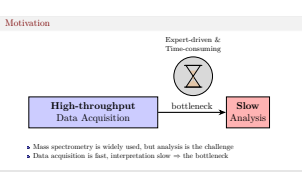


Let me begin with a brief outline of the talk.
I will start with the problem statement and the background on mass spectrometry. I then turn to the data, and to how synthetic data is generated. This is followed by the machine-learning model and the results. I will close with the conclusions I draw from the work.

Motivation

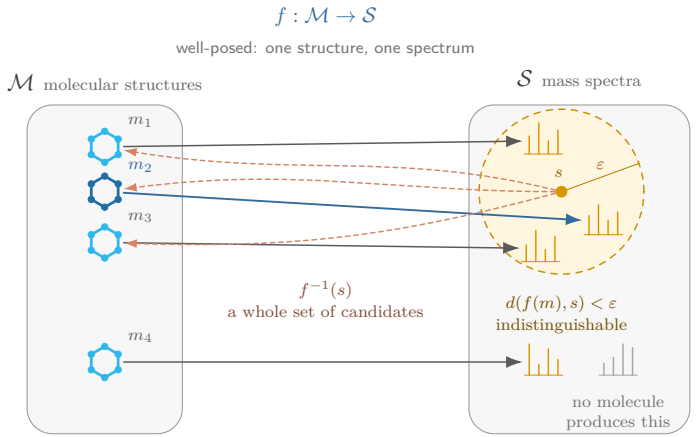


- Mass spectrometry is widely used, but analysis is the challenge
- Data acquisition is fast, interpretation slow ⇒ the bottleneck

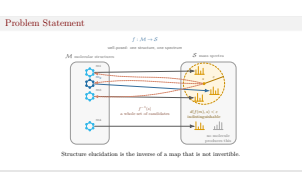


What is the motivation?
Mass spectrometry is among the most widely used techniques for analyzing molecules. Interpretation of the data for novel molecules, however, is still largely manual and expert-driven, and correspondingly slow.
Data acquisition is fast — and none of that speed carries over into the analysis. That asymmetry is where the bottleneck sits.
Therefore, developing computational methods in structure elucidation is a worthwhile goal.

Problem Statement

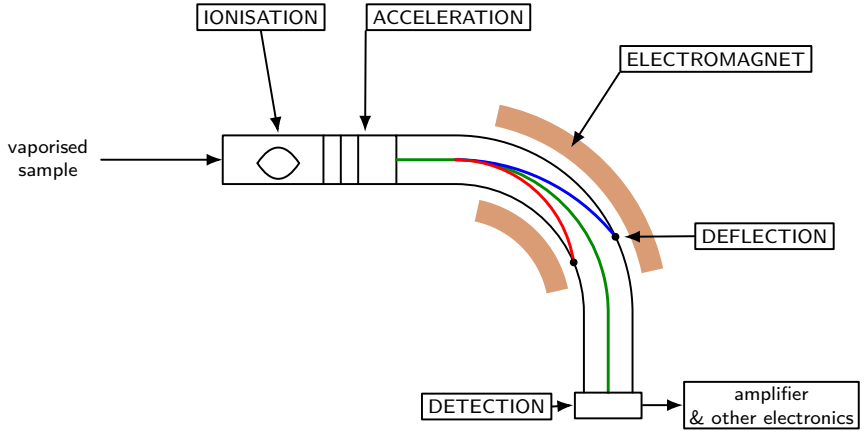


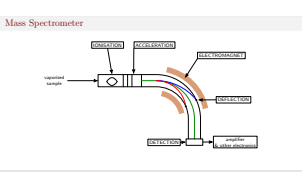
Structure elucidation is the inverse of a map that is not invertible.



Stated formally, the problem is a map between two spaces. On the left, the space of molecular structures. On the right, the space of mass spectra.
Under fixed measurement conditions, the forward direction is well behaved: one structure gives one spectrum.
What I am ultimately after is the other direction, so-called structure elucidation. Given a spectrum, recover the structure.
But that map is not invertible.
The shaded region is everything within epsilon of the measured spectrum. Several different molecules land inside it: their spectra do differ, but by less than the tolerance.
Here the inverse is the red arrow. What matches a given spectrum, within epsilon, is not one molecule. It is a whole set of candidates.

Mass Spectrometer





A brief reminder of the measurement principle.
We start with a sample, which is first ionized and accellerated into the mass analyzer.
There the ions are then separated according to their mass-to-charge ratio, and are finally registered at a detector — which is what yields the spectrum.
The point that matters for this work: under electron ionization the molecule does not remain intact, but fragments. The spectrum is therefore a fingerprint of those fragments, and it is precisely that structure I exploit later on. So the spectrum can be related to certain molecular structures.

Modelling Mass Spectra – An Open Problem

- **Rule-based** but limited to rule set
- **Data-driven** models do not generalize
- **Ab initio** does not scale
- currently <20% of detected compounds are identified

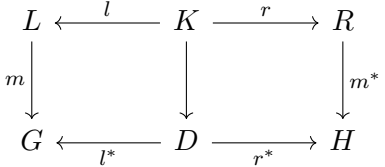
Source: [1, 2, 3, 4]

Modelling Mass Spectra – An Open Problem
- **Rule-based** but limited to rule set
- **Data-driven** models do not generalize
- **Ab initio** does not scale
- currently <20% of detected compounds are identified

Before I continue, one point on the state of the art.
Modelling mass spectra is not a solved problem.
There are three families of approaches, and each trades something away. Rule-based in silico fragmentation is interpretable, but it only ever sees what the rule set encodes. Data-driven models do well on molecules that resemble their training data, but they carry no mechanism and do not generalize on unseen structures. Ab initio calculations are mechanistically faithful, but far too expensive to run at a big scale.
So in practice four out of five compounds an instrument detects are never named — that is the size of the problem, and why even a partial improvement is worth having.
So this is open ground — which is what motivates the route I take next.

MØD – A Graph Transformation Tool

- Molecules as labelled graphs, reactions as *rewrite rules*
- Repeated application builds the *derivation graph* of a reaction network
- A rule is a span $L \leftarrow K \rightarrow R$:
  - L – what must be present
  - K – what is preserved (context)
  - R – what it becomes



Host graph $G \rightarrow$ product graph H

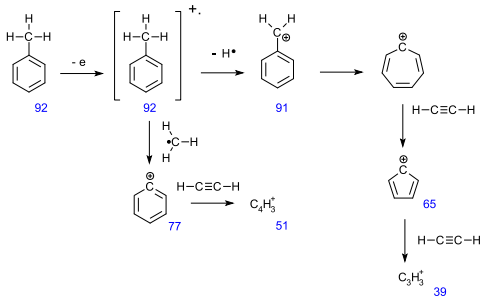
Source: [5, 6]

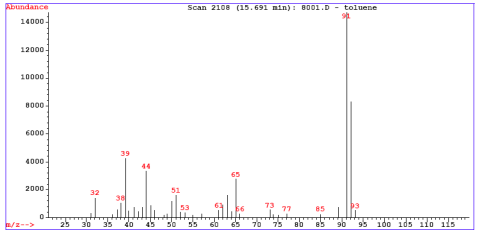
All of the fragmentation chemistry in this work is expressed as graph transformation, and the tool that carries it out is MÖL. MÖL was developed, in the Theoretical Biochemistry Group — among its authors is my supervisor, Christoph Flamm.
In one sentence: a molecule is a labelled graph, a reaction is a rule that rewrites part of that graph, and applying rules over and over gives the derivation graph — the network of everything reachable from the starting molecule.
A rule has three parts: the left-hand side is the pattern that must be present, the interface is the context that is kept, and the right-hand side is what it becomes.

And that is the end of the background. Everything from here on — the rule encoding, the predicates, the data generation, the model and the evaluation — is my own work.

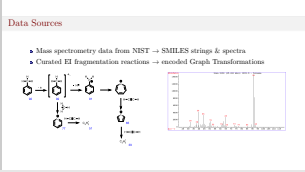
Data Sources

- Mass spectrometry data from NIST → SMILES strings & spectra
- Curated EI fragmentation reactions → encoded Graph Transformations





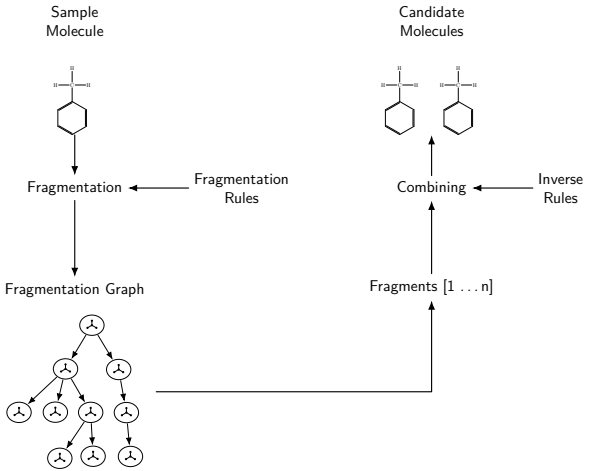
Source: [7, 8]

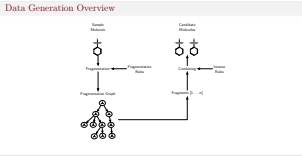


Two data sources enter the work.
The first is mass-spectrometry data from NIST. It is considerd the gold standard reference library for electron-ionization spectra. It provides the structures, as SMILES strings, together with the measured spectra.
The second source is a set of curated EI fragmentation reactions from the literature, which I have encoded as Graph Transformations. On the left is the fragmentation of toluene, on the right the corresponding spectrum. I have only encoded 153 reactions, so the coverage is limited.

The corpus is only 66 molecules, due to the high computational cost of my unoptimized code.

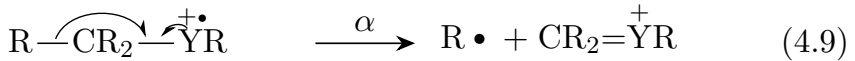
Data Generation Overview





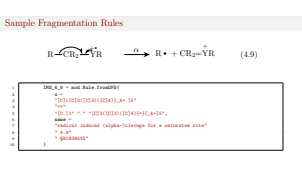
This figure summarizes the entire pipeline of the synthetic data generation.
A molecule is fragmented by applying the rule set in a Graph Transformation Tool.
That yields a fragmentation graph, from which the fragments are read off.
The reverse direction then **recombines** those fragments into theoretically possible structures, using the inverted reaction rules.
This is how candidate structures for a given spectrum are obtained.

Sample Fragmentation Rules



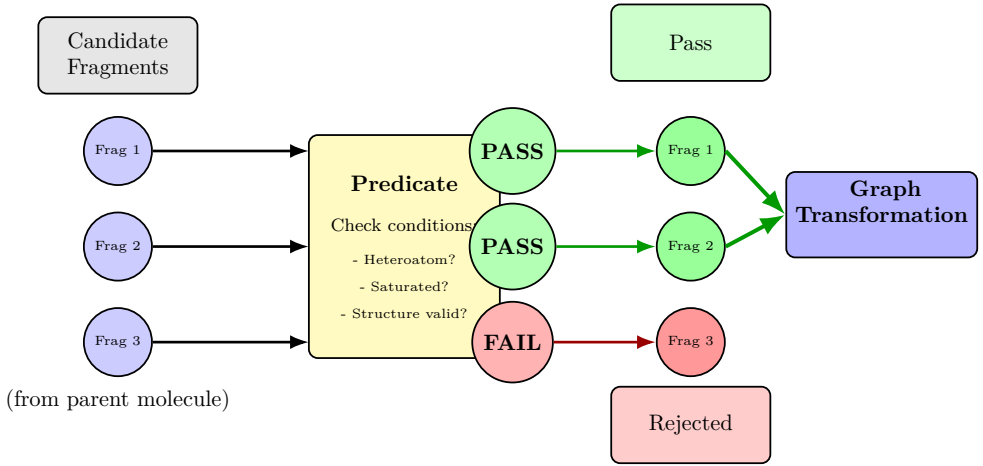

```
1   IMS_4_9 = mod.Rule.fromDFS(
2       s =
3       "[C]1[C]2([C]3)([C]4)[_A+.]5"
4       ">>"
5       "[C.]1" "." "[C]2([C]3)([C]4){=}[_A+]5",
6       name =
7       "radical induced (alpha-)clevage for a saturated site"
8       " 4.9"
9       " §R1R3R4Y5"
10  )
```

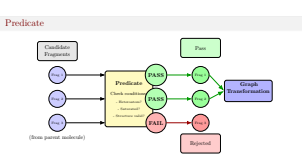
Source: [9]



At the top you see one of these rules in its original, literature form.
The essential observation is that the formulation is not restricted to individual atoms. It also quantifies over entire substituent groups. Those are the R and Y placeholders, for alkyl and heteroatom groups.
Below is the same rule in code. The graph transformation itself operates on single atoms only. So I carry the group conditions in the rule name, after the paragraph sign. Here atoms 1, 3 and 4 are required to be alkyl groups, and atom 5 a heteroatom group.
On the following slides you will see how this notation is enforced during the transformation.

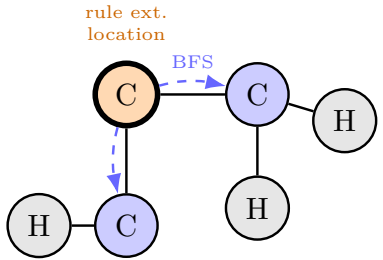
Predicate





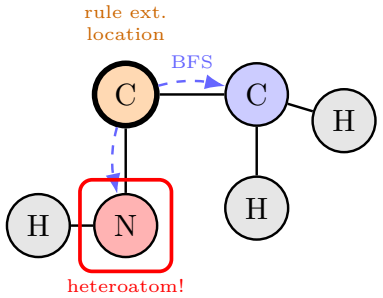
MÖL provides an interface for so-called predicates. Predicates are callbacks that are evaluated during the transformation. They are here used to filter candidate matches according to the rule extension. The next few slides discuss the ones I implemented.

Predicate – Alkyl Group



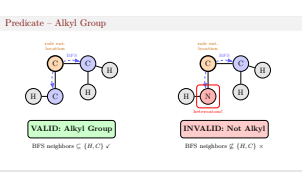
VALID: Alkyl Group

BFS neighbors $\subseteq \{H, C\}$ ✓



INVALID: Not Alkyl

BFS neighbors $\not\subseteq \{H, C\}$ ×

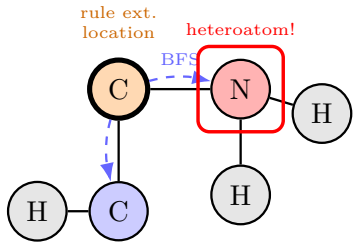


The first is the alkyl group. An alkyl substituent consists of nothing but carbon and hydrogen.

I start at the atom at which the rule is defined. From there I collect the neighborhood breadth-first. The search terminates at atoms that already belong to the match of the original rule.

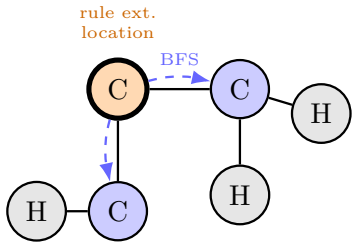
The condition is then simply whether every collected atom is a carbon or a hydrogen. If so, the group is accepted as alkyl; otherwise the match is rejected.

Predicate – Heteroatom Group



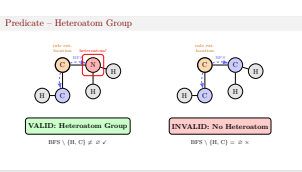
VALID: Heteroatom Group

BFS \ {H, C} ≠ ∅ ✓



INVALID: No Heteroatom

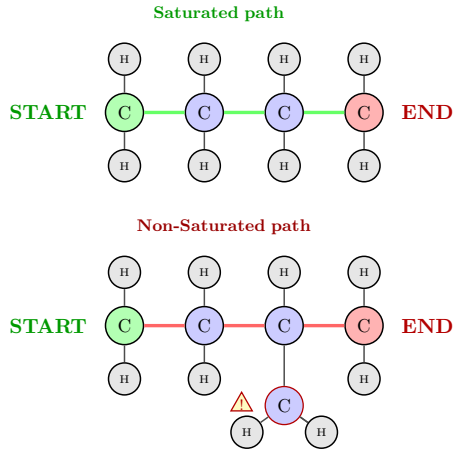
BFS \ {H, C} = ∅ ×

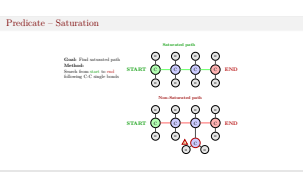


The heteroatom group is the complementary condition. Here at least one atom is required to be *neither* carbon *nor* hydrogen.
Again I collect the neighborhood breadth-first. Only atoms the original rule has not matched are considered.
The set is then tested for any element outside hydrogen and carbon.
If such an atom is present, the group is accepted. If the set is hydrogen and carbon only, the match is rejected.

Predicate – Saturation

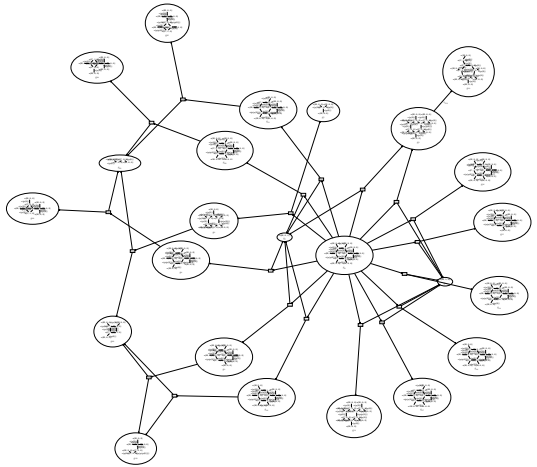
Goal: Find saturated path
Method:
Search from start to end following C-C single bonds





The third predicate is saturation. The requirement is a chain of carbon atoms connected exclusively by single bonds, in which every carbon carries hydrogen substituents only.
The search proceeds from the start atom to the end atom, along carbon–carbon single bonds. The upper example satisfies the condition. The lower example does not, because of the branched carbon.

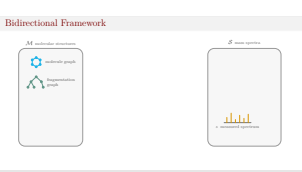
Sample Fragmentation Graph



Sample Fragmentation Graph

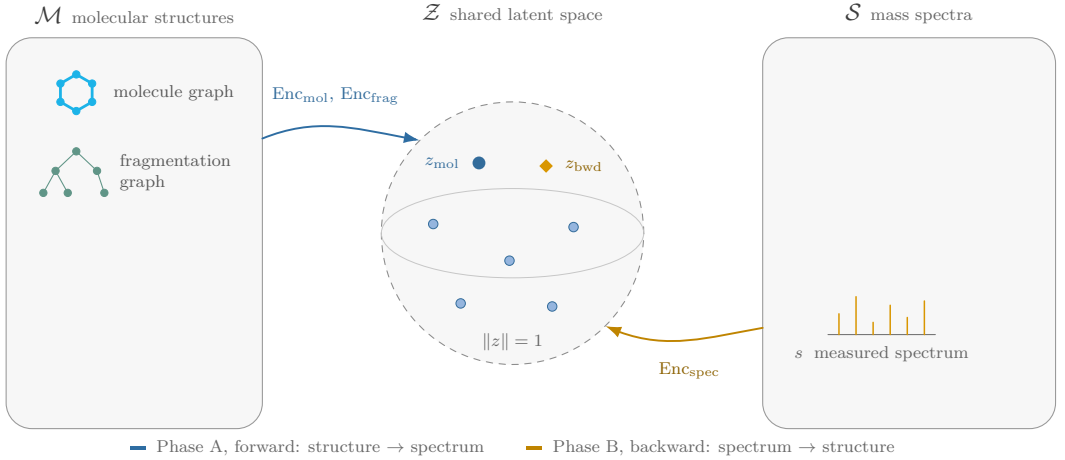
This is what the fragmentation produces, drawn as a hypergraph.
The vertices are the fragments, and the edges are the reactions relating them. The content of the individual nodes is not meant to be read here.
The relevant point for the talk is the structure: a single input molecule gives rise to a connected space of fragments, and that space is what the model gets to work with.

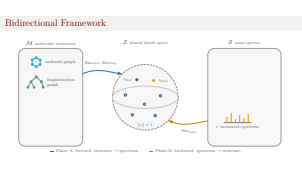
Bidirectional Framework



Here is the problem slide again: molecules on the left, spectra on the right. For each molecule I keep two views side by side — its molecule graph, and the fragmentation graph the derivation produces. On the right, the measured spectrum.

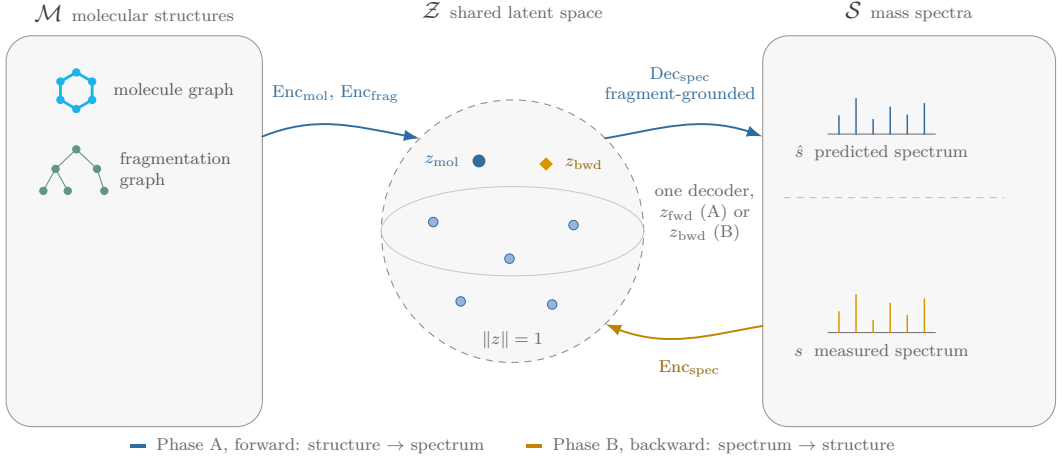
Bidirectional Framework

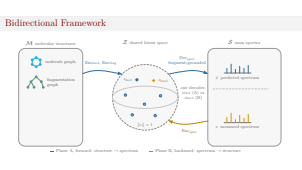




What's new is the middle — one shared latent space that both sides are encoded into. Both encoders map straight in: the molecule graph, the fragmentation graph, and the spectrum. The dots are the index: every known molecule already has a place in here. Highlighted is one matched pair — a molecule's latent and its own spectrum's latent.

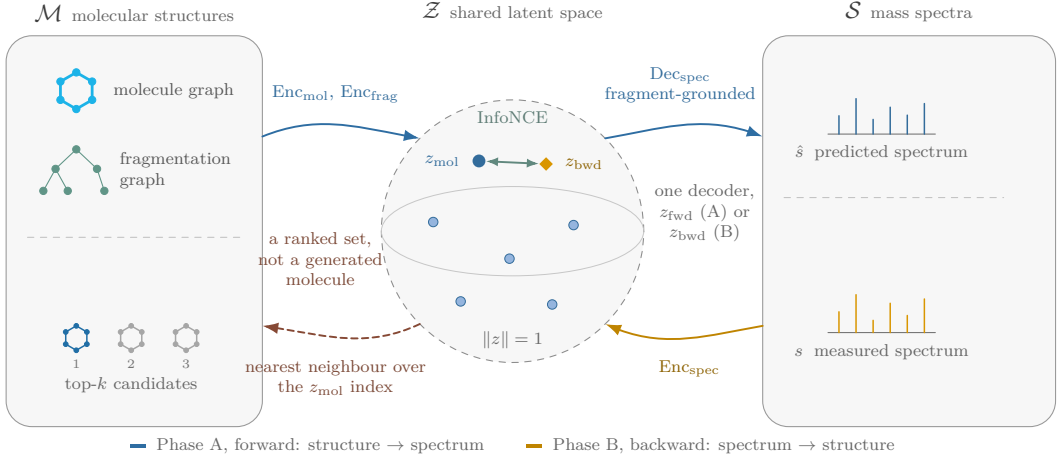
Bidirectional Framework

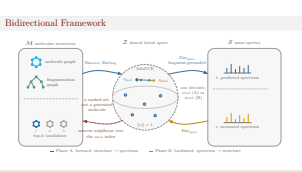




Going clockwise over the top is Phase A, the forward direction. The decoder does not emit a free-form spectrum, it predicts one intensity per fragment. So every peak sits at a chemically realisable mass by construction. And it is the same decoder either way — fed a molecule's own latent here in Phase A, or a spectrum's latent in Phase B.

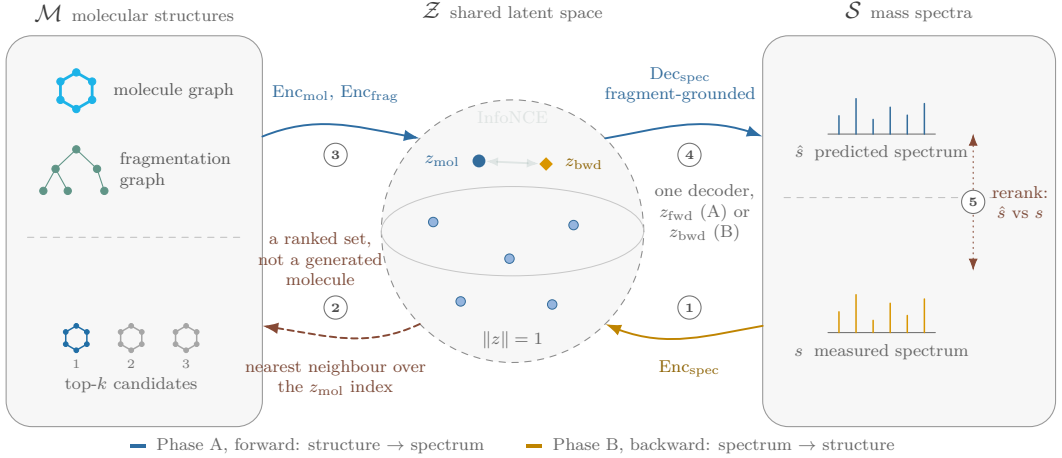
Bidirectional Framework

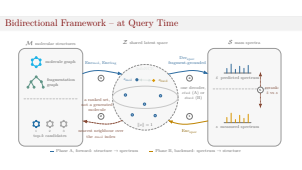




Coming back underneath is Phase B, the backward direction: nearest-neighbour retrieval over that same molecule index. What comes back is a *ranked set* of known structures, not a generated molecule. That green tie in the middle is InfoNCE. During training it pulls each spectrum latent onto its own molecule. And it pushes away from the others in the batch, which is what makes the two directions share one space rather than two.

Bidirectional Framework – at Query Time





At query time only part of this runs, in the order numbered here: encode the measured spectrum, look up its nearest molecule latents in the index, run those candidates forward through their own encoders, decode each one's predicted spectrum, then rerank by comparing it against what we measured. Retrieval proposes, the forward model checks.

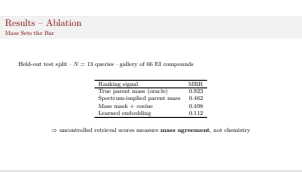
Results – Ablation

Mass Sets the Bar

Held-out test split · $N = 13$ queries · gallery of 66 EI compounds

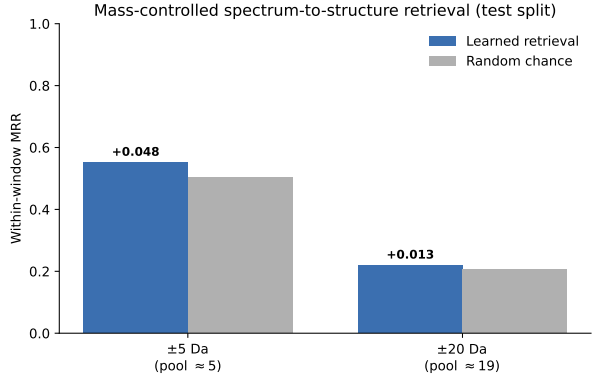
Ranking signal	MRR
True parent mass (oracle)	0.923
Spectrum-implied parent mass	0.462
Mass mask + cosine	0.498
Learned embedding	0.112

$\Rightarrow$ uncontrolled retrieval scores measure **mass agreement**, not chemistry



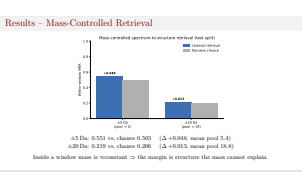
I now turn to the results. I start with the observation that determines how every other number has to be read.
With a gallery of only 66 compounds, parent mass almost identifies the molecule on its own.
An oracle that ranks candidates by their true mass attains a mean reciprocal rank of 0.92.
Even a mass estimate read off the heaviest peak attains 0.46.
The learned embedding, scored on the full gallery, attains 0.11.
So none of these numbers beat mass.
The reranker combines a mass mask with the forward model. Drop the forward model and keep just the mask — still 0.50.
So the question is not how high the retrieval score is. It is whether anything remains once mass is controlled for. We will see that on the next slide.

Results – Mass-Controlled Retrieval



±5 Da: 0.551 vs. chance 0.503 (Δ +0.048, mean pool 5.4)
±20 Da: 0.219 vs. chance 0.206 (Δ +0.013, mean pool 18.8)

Inside a window mass is ≈constant ⇒ the margin is structure the mass cannot explain.

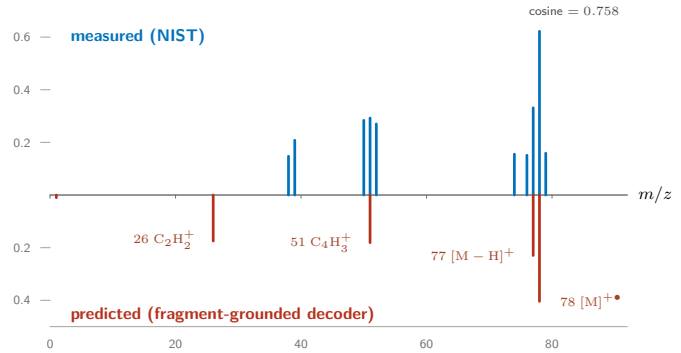


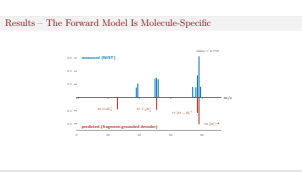
Here mass is held fixed. I restrict the ranking to candidates within a window around the query's true mass. Plus or minus 5 Dalton, and plus or minus 20. Within such a window every candidate has essentially the same mass. So mass can no longer account for the ranking.

I measure against the analytic chance level for a pool of that size. In the narrow window the learned retrieval attains 0.551, against 0.503. In the wide window, 0.219 against 0.206. That is a margin of roughly five and one percentage points.

What is important is that they are positive on molecules seen neither in training nor in model selection. And they are the only part of the evaluation that mass cannot explain.

Results – The Forward Model Is Molecule-Specific





One more result, and this one concerns the forward model rather than retrieval.
Unconstrained spectrum decoders have a well-known failure mode: they predict roughly the dataset-average spectrum for every input, and a reranker on top only appears to help, because the help comes from the mass filter. Mine cannot do that, by construction — every predicted intensity is scattered from fragments derived for the input molecule.
The figure shows benzene, measured upward and predicted mirrored downward. Predicted peaks fall at 78, the molecular ion, 77 for hydrogen loss, and 51 for a fragment, and all three appear in the measurement. A fourth predicted peak sits at 26, which the rules derive but the measurement does not show. Vice versa, the measured spectrum has a peak at 39, which the rules do not derive and the model does not predict.
Both misses have an explanation, and they differ in kind. The peak at 26 may well be right and simply unobservable: EI spectra are usually not recorded below about mass-to-charge 35. The peak at 39 is the opposite case — a real peak my rules do not reach.
Corpus-wide, fifty-five percent of the intensity the rules cannot explain sits on even-electron ions, the signature of hydrogen transfer rather than simple cleavage.

The rules account for about forty-two percent of measured peak intensity. So the rules are a ceiling: the model can only work with the part of the spectrum they reach, and no amount of training moves that bound.

Conclusions

Contributions

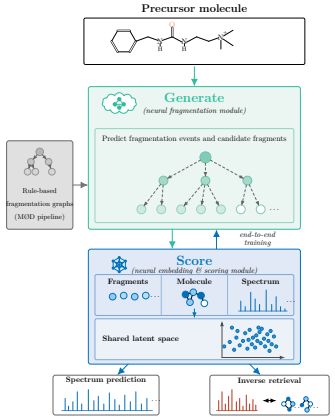
- **Fragment-grounded forward model**: intensities scattered from the molecule's own derived fragments ⇒ molecule-specific *by construction*
- **Mass-controlled evaluation**: separates learned structure from parent-mass filtering

Supporting work: bidirectional framework ($f : \mathcal{M} \to \mathcal{S}$, $f^{-1} : \mathcal{S} \to \mathcal{M}$) + rule-extension DSL

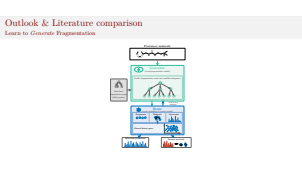
To summarize. There are two contributions I would defend.
First, the forward model is fragment-grounded. Every predicted intensity is scattered from the molecule's own derived fragments. So it is molecule-specific by construction — a design property, not an empirical accident.
Second, and more importantly, the evaluation. Mass is so strong a signal here that most retrieval figures amount to mass filtering. The mass-controlled protocol separates the two.
Around those sit the bidirectional framework and the rule-extension DSL.

Outlook & Literature comparison

Learn to *Generate* Fragmentation



Source: [10]



A structural design premise of this thesis: the model encodes precomputed fragmentation graphs. It does not learn fragmentation itself.

The principal inspiration for this is ICEBERG, by Goldman and colleagues. ICEBERG divides the task in two. A Generate module proposes likely fragmentation events. A Score module assigns the intensities.

Viewed through that lens, my model already provides the Score side — the lower half of the figure. It embeds molecules, fragments and spectra into a shared latent space, and produces both the forward prediction and the inverse retrieval.

What is missing is the Generate module upstream, the component that learns to propose fragmentation. The symbolic fragmentation graphs could serve as proxy supervision for training exactly that.

The two would then be trained end-to-end. The model would learn not only how to use fragments, but which fragmentation structures to construct in the first place.

References

[1] Adamo Young, Hannes Röst, and Bo Wang. "Tandem mass spectrum prediction for small molecules using graph transformers". In: *Nature Machine Intelligence* 6.4 (Apr. 2024), pp. 404–416. ISSN: 2522-5839. DOI: 10.1038/s42256-024-00816-8.

[2] Yannek Nowatzky et al. "FIORA: Local neighborhood-based prediction of compound mass spectra from single fragmentation events". In: *Nature Communications* 16.1 (Mar. 2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-57422-4.

[3] Mehdi B. Hamaneh et al. "Systematic Assessment of Deep Learning-Based Predictors of Fragmentation Intensity Profiles". In: *Journal of Proteome Research* 23.6 (May 2024), pp. 1983–1999. ISSN: 1535-3907. DOI: 10.1021/acs.jproteome.3c00857.

[4] Ivana Blaženović et al. "Software Tools and Approaches for Compound Identification of LC-MS/MS Data in Metabolomics". In: *Metabolites* 8.2 (May 2018), p. 31. ISSN: 2218-1989. DOI: 10.3390/metabo8020031.

[5] Jakob L. Andersen et al. "An intermediate level of abstraction for computational systems chemistry". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 375.2109 (Nov. 2017), p. 20160354. ISSN: 1471-2962. DOI: 10.1098/rsta.2016.0354.

[6] MØD Developers. *Derivation Graph Strategy (dgStrat) — MØD Documentation*. Accessed 2026-03-24. 2022. URL: https://imada.sdu.dk/u/daniel/DM840-2022/Material/mod-doc/dgStrat/index.html (visited on 03/24/2026).

[7] V8rik. *Toluene Fragmentation*. 2008. URL: https://upload.wikimedia.org/wikipedia/commons/6/6c/TolueneFragmentation.svg (visited on 01/20/2026).

[8] Paginazero, Ronhjones. *Toluene Spectra*. 2017. URL: https://upload.wikimedia.org/wikipedia/commons/2/2c/Spettro_di_massa_del_toluene.PNG (visited on 01/20/2026).

[9] Fred W McLafferty and František Tureček. *Interpretation von Massenspektren*. ger. Spektrum Lehrbuch. Berlin [u.a.]: Spektrum Akad. Verl., 1995. ISBN: 9783642398483.

[10] Samuel Goldman, Janet Li, and Connor W. Coley. "Generating Molecular Fragmentation Graphs with Autoregressive Neural Networks". In: *Analytical Chemistry* 96.8 (Feb. 2024), pp. 3419–3428. ISSN: 1520-6882. DOI: 10.1021/acs.analchem.3c04654.


References

These are the sources and references used in this presentation.

Discussion

Questions?

That concludes the presentation of my thesis.
Thank you for your attention.
I would be happy to take your questions.

Discussion
Questions?

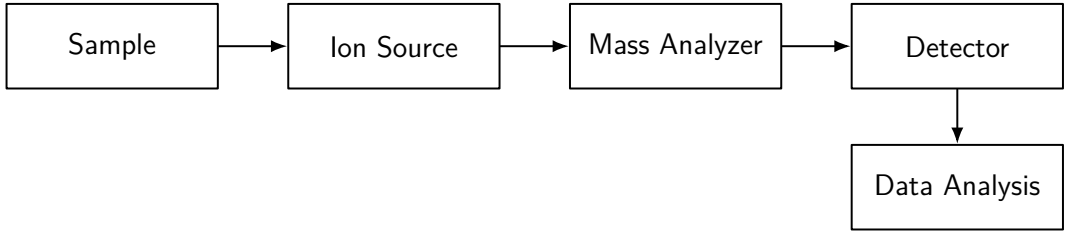
Supplementary Figures

- Instrumentation & ionization methods
- Molecular representation
- Model architecture — training & inference
- The residual MLP block
- Latent-space alignment (InfoNCE)
- Decoder conditioning
- Spectral comparison measures
- Data-generation cost
- What the speedup enabled
- Train / validation / test split

Supplementary Figures
- Instrumentation & ionization methods
- Molecular representation
- Model architecture — training & inference
- The residual MLP block
- Latent-space alignment (InfoNCE)
- Decoder conditioning
- Spectral comparison measures
- Data-generation cost
- What the speedup enabled
- Train / validation / test split

These are backup slides — figures from the thesis that did not enter the talk itself. I will go to whichever one is relevant to your question.

Mass Spectrometer – Block Diagram

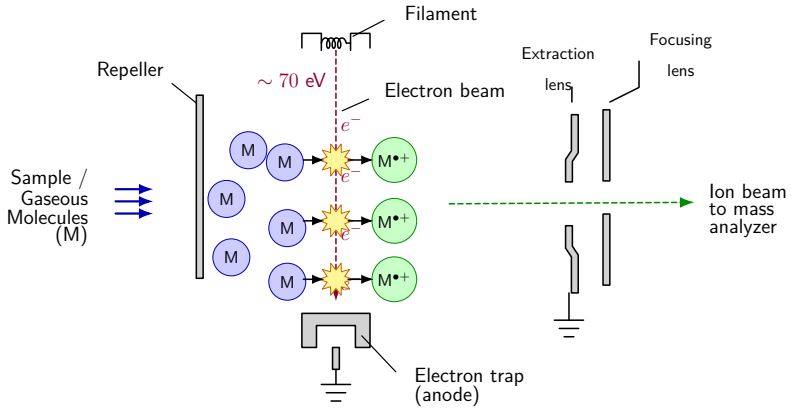


Inlet → ion source → mass analyzer → detector → data analysis.

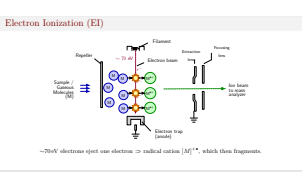
Mass Spectrometer – Block Diagram
Sample
Ion Source
Mass Analyzer
Detector
Data Analysis
Inlet → ion source → mass analyzer → detector → data analysis.

The instrument as a block diagram: sample in, ionization, separation by m/z, detection, data analysis.

Electron Ionization (EI)

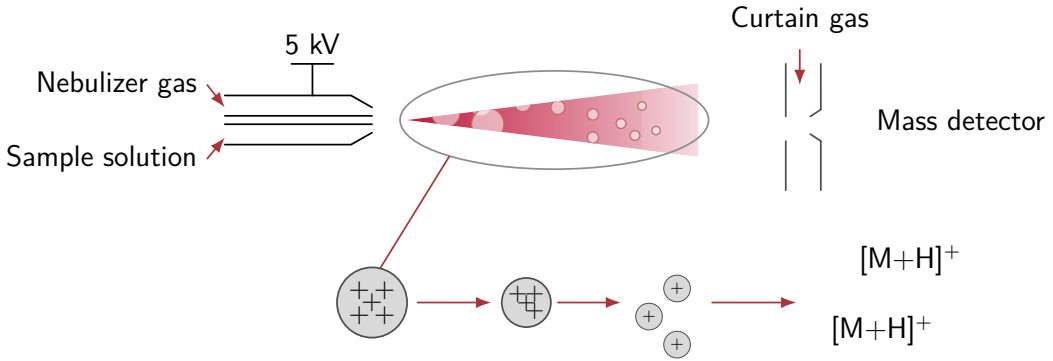


$\sim$70 eV electrons eject one electron $\Rightarrow$ radical cation $[M]^{+\bullet}$, which then fragments.

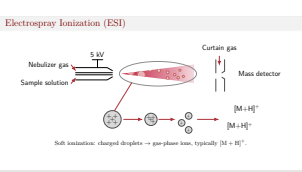


This is the ionization method the entire thesis is concerned with. A 70-electron-volt beam ejects one electron, producing a radical cation whose excess internal energy is what drives the fragmentation.

Electrospray Ionization (ESI)

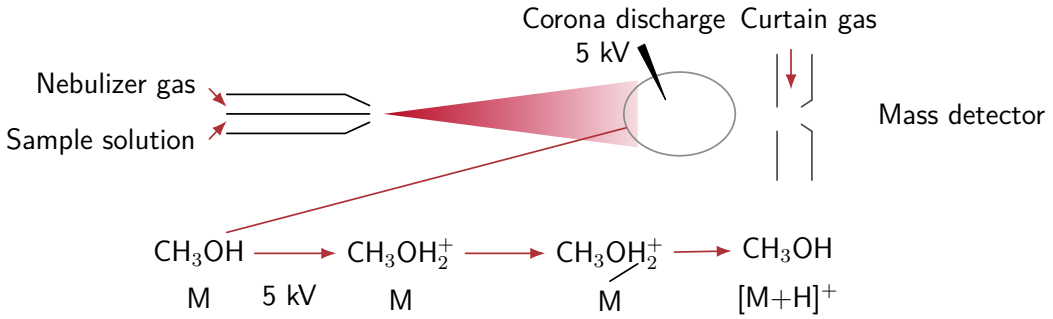


Soft ionization: charged droplets → gas-phase ions, typically $[M + H]^+$.

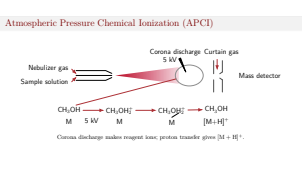


ESI for contrast: a soft method, which largely preserves the intact molecule and produces very little fragmentation — which is precisely the information my approach relies on.

Atmospheric Pressure Chemical Ionization (APCI)

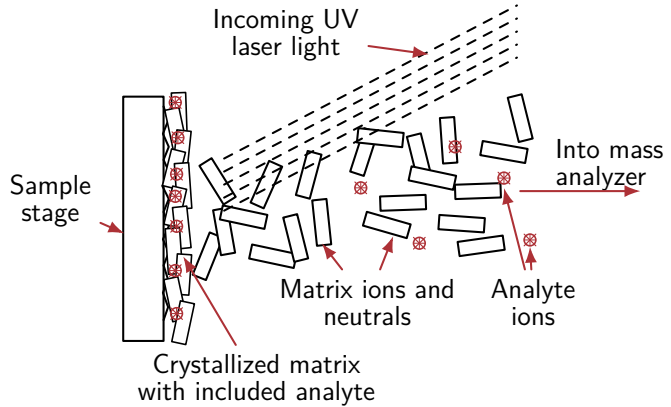


Corona discharge makes reagent ions; proton transfer gives $[M + H]^+$.

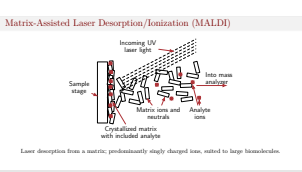


APCI: the solvent is ionized first by a corona discharge, and the resulting reagent ions then transfer a proton to the analyte.

Matrix-Assisted Laser Desorption/Ionization (MALDI)

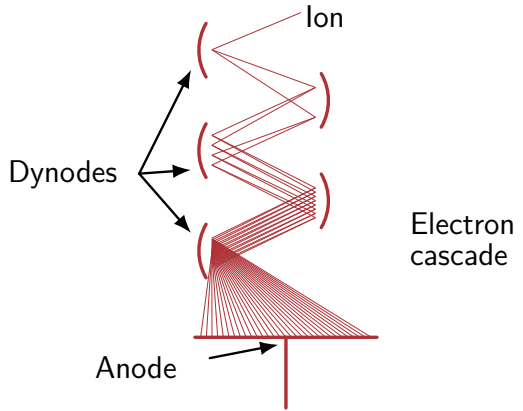


Laser desorption from a matrix; predominantly singly charged ions, suited to large biomolecules.



MALDI: the analyte is embedded in a matrix, a laser pulse desorbs both, and the matrix ions effect the ionization. The ions produced are predominantly singly charged, and the method scales to large biomolecules.

Electron Multiplier Detector

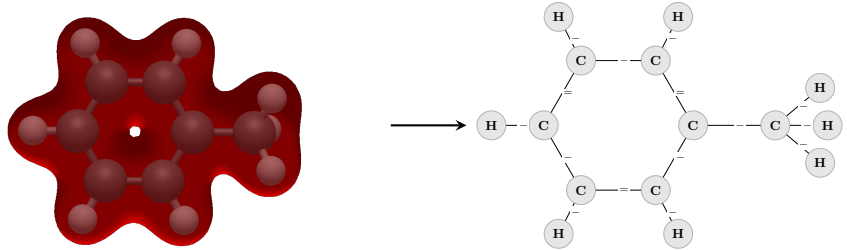


Cascading secondary electrons across dynodes amplify a single ion into a measurable current pulse.

Electron Multiplier Detector
Ion
Dynodes
Electron cascade
Anode
Cascading secondary electrons across dynodes amplify a single ion into a measurable current pulse.

And the detection side: a single ion strikes the first dynode, each stage multiplies the electrons, and after several stages the result is a current pulse that can be digitized.

Molecule as a Graph

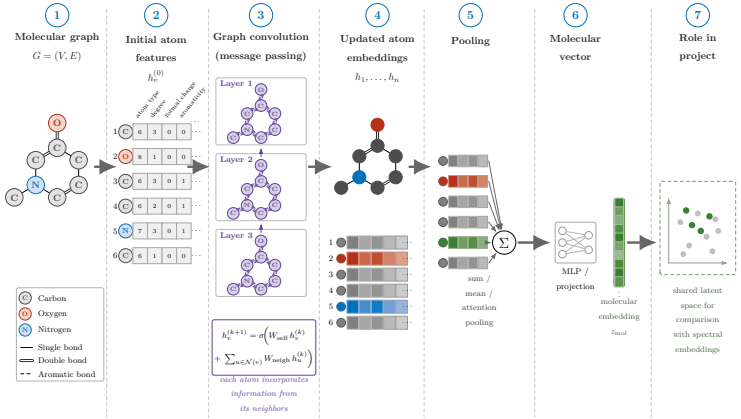


Geometry and electron density are abstracted away; connectivity is kept.

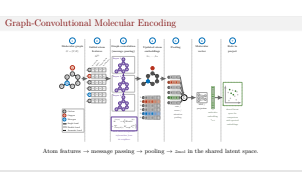
Molecule as a Graph
Geometry and electron density are abstracted away; connectivity is kept.

The abstraction on which everything rests: toluene as a quantum-mechanical object on the left, as a graph on the right. Atoms become vertices, bonds become edges, while geometry and electron density are discarded.

Graph-Convolutional Molecular Encoding

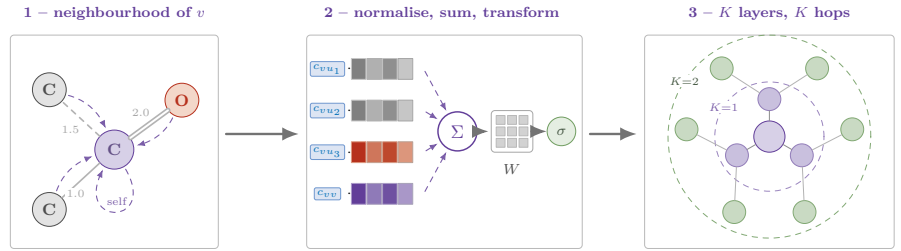


Atom features → message passing → pooling → z_{mol} in the shared latent space.



The molecule encoder in detail. Each atom is initialized with a feature vector, message passing renders those embeddings context-aware, pooling collapses them into one fixed-length vector, and a projection maps it into the shared latent space.

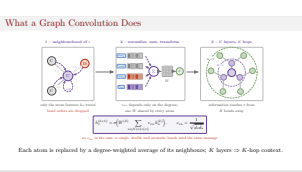
What a Graph Convolution Does



$$h_v^{(k+1)} = \sigma\Big(W^{(k)} \sum_{u \in \mathcal{N}(v)\cup\{v\}} c_{vu}\, h_u^{(k)}\Big), \qquad c_{vu} = \frac{1}{\sqrt{\hat{d}_v \hat{d}_u}}$$

no e_{uv} in the sum ⇒ single, double and aromatic bonds send the same message

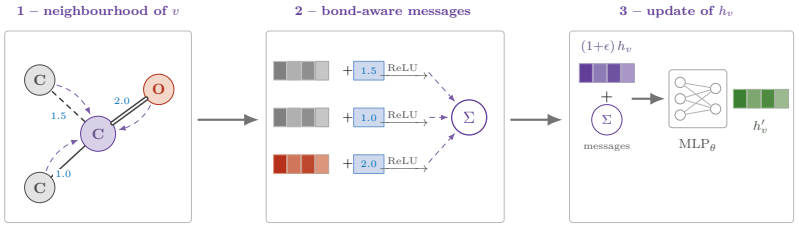
Each atom is replaced by a degree-weighted average of its neighbours; K layers ⇒ K-hop context.



The baseline convolution, for completeness — this is the Kipf–Welling GCN, and it is what the very first version of my encoder used.
One layer does three things. Every atom collects the feature vectors of its neighbours, itself included via a self-loop. The contributions are weighted by a coefficient that depends only on the two degrees, so a highly connected atom does not dominate. The weighted sum is passed through one weight matrix shared by all atoms and a non-linearity. Stacking K layers means every atom has seen everything within K bonds.
The point of the middle panel is the coefficient: it is fixed by the graph structure, not learned, and it is a normalized average. An average cannot count — it cannot tell one neighbour of a kind from three.

And the point of the first panel is what is missing. The bond never enters the sum. Only atom features travel along the edges, so a single, a double and an aromatic bond deliver exactly the same message. After pooling, what is left is essentially which atoms are present — the composition, and therefore the mass. That is the confound the results chapter is about, and it is why the next slide moves to GINE.

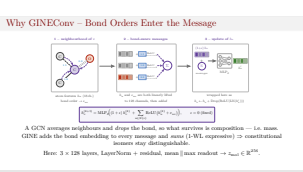
Why GINEConv – Bond Orders Enter the Message



$$h_v^{(k+1)} = \mathrm{MLP}_\theta\Big((1+\epsilon)\,h_v^{(k)} + \sum_{u \in \mathcal{N}(v)} \mathrm{ReLU}\big(h_u^{(k)} + e_{uv}\big)\Big), \qquad \epsilon = 0 \text{ (fixed)}$$

A GCN averages neighbours and *drops* the bond, so what survives is composition — i.e. mass. GINE adds the bond embedding to every message and *sums* (1-WL expressive) ⇒ constitutional isomers stay distinguishable.

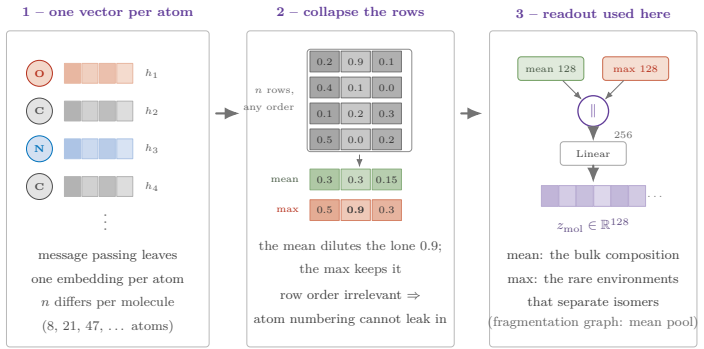
Here: 3×128 layers, LayerNorm + residual, mean $\|$ max readout $\rightarrow z_{\mathrm{mol}} \in \mathbb{R}^{256}$.



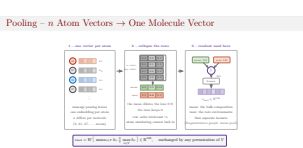
A closer look at the convolution itself, since the choice is not incidental.
A plain graph convolution averages the neighbouring atom features. The bond never enters the update, so what survives pooling is essentially which atoms are present — the composition, and with it the mass. That is exactly the confound the results chapter is fighting.
GINE differs in two ways. First, the bond order is lifted to the same width as the atom features and *added* into each message, so a double bond and a single bond produce different messages. Second, the aggregation is a sum followed by an MLP rather than a mean, which is what makes the architecture as expressive as the one-dimensional Weisfeiler–Leman test. A mean cannot see multiplicity; a sum can.
Concretely: eighteen atom channels — element one-hot, charge, radical, degree, aromatic flag, incident bond order — and one bond channel, both projected to 128. Three layers, each with LayerNorm, ReLU and a residual connection. Readout is mean concatenated with max, because the max keeps the few atom environments that separate isomers, which a mean over all atoms dilutes.

The practical consequence: my first encoder was a two-layer GCN over atom identity alone, and it collapsed to composition. This one does not.

Pooling – n Atom Vectors → One Molecule Vector



$$z_{\text{mol}} = W\left[\ \text{mean}_{v\in V}\, h_v \ \Big\|\ \max_{v\in V} h_v\ \right] \in \mathbb{R}^{128}, \quad \text{unchanged by any permutation of } V$$



"Pooling" on the training slide is this step, and it is the only place where the size of the molecule disappears. Message passing gives one vector per atom, so a molecule with eight atoms leaves eight vectors and one with forty-seven leaves forty-seven. Everything downstream — the latent space, the contrastive loss — needs a single vector of fixed width. Pooling is the operator that collapses the set.

The constraint is that it must not depend on the order of the rows. Atom numbering is an artefact of the file, not chemistry, so the readout has to be a symmetric function: sum, mean, or max.

The middle panel is why the choice among them matters. The mean of that column is 0.3 — the single atom carrying 0.9 is averaged away by the three that carry almost nothing — while the max returns 0.9. The dilution goes as one over the atom count, and the environments that distinguish constitutional isomers are exactly the rare ones.

So the encoder concatenates both: 128 channels of mean, 128 of max, and one linear layer projects the 256 down to the 128-dimensional latent. The mean carries the bulk composition, the max carries the outliers.

The fragmentation-graph encoder uses a plain mean pool — there the vertices are fragments, and there is no isomer distinction to preserve at that level.

Two-Level Representation – Overview

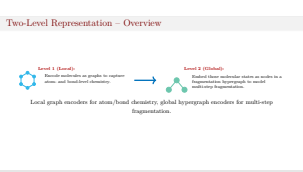
Level 1 (Local):

Encode molecules as graphs to capture atom- and bond-level chemistry.

Level 2 (Global):

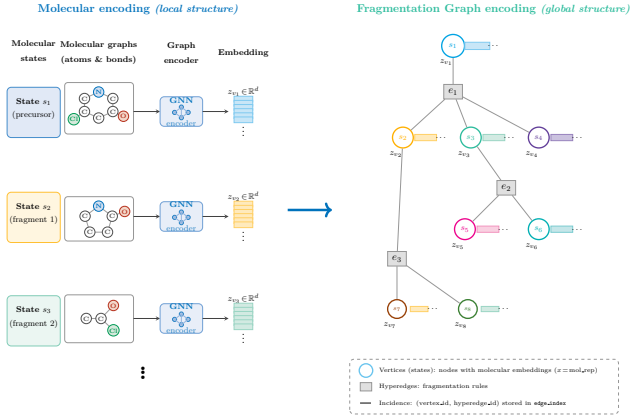
Embed those molecular states as nodes in a fragmentation hypergraph to model multi-step fragmentation.

Local graph encoders for atom/bond chemistry, global hypergraph encoders for multi-step fragmentation.

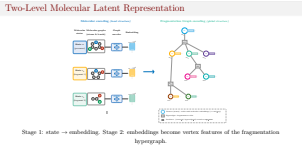


The two-level construction in one figure: the lower level encodes the chemistry within each molecular state, the upper level encodes how those states are related by fragmentation.

Two-Level Molecular Latent Representation

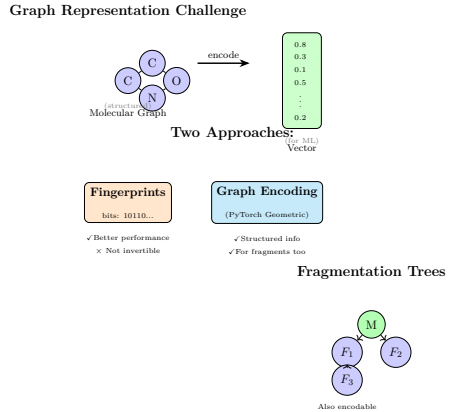


Stage 1: state → embedding. Stage 2: embeddings become vertex features of the fragmentation hypergraph.

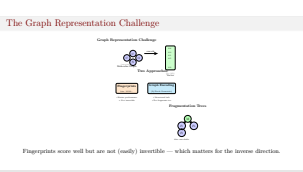


And the same construction stated concretely. Each molecular state is first encoded individually. Those embeddings then enter as vertex features of the fragmentation hypergraph, whose hyperedges carry the multi-step structure on top.

The Graph Representation Challenge

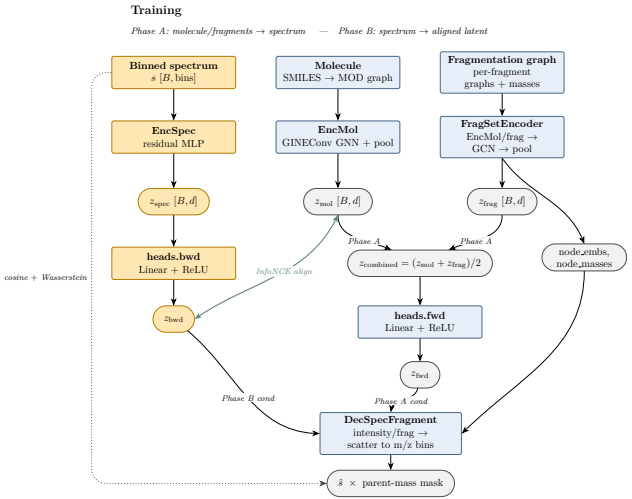


Fingerprints score well but are not (easily) invertible — which matters for the inverse direction.



Why the representation question is not settled: fingerprints perform very well, but they cannot readily be inverted back to a structure — and the inverse direction is exactly what is required here.

Architecture – Training



Now to the machine-learning model. This is the architecture, trained in two phases — Phase A and Phase B.

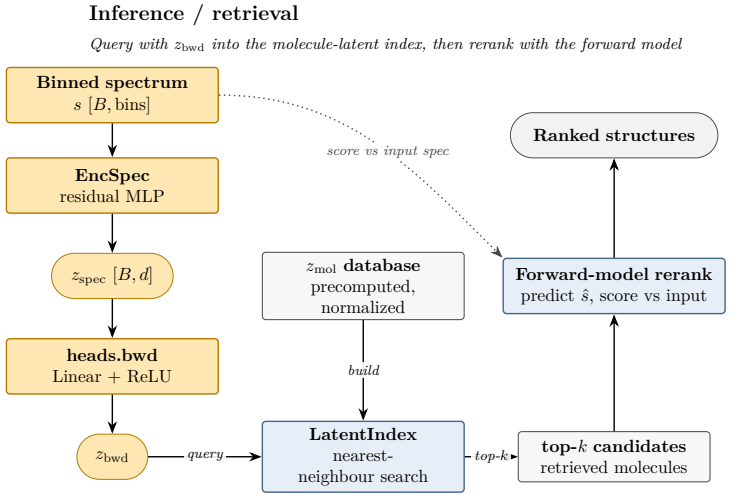
Three things go in: the measured spectrum, the molecule graph and the fragmentation graph. Each has its own encoder, and each produces a vector of the same dimension. A GINEConv network with pooling encodes the molecule. A two-level encoder handles the fragmentation graph: first each fragment, then a GCN over the whole graph. A residual MLP encodes the spectrum.

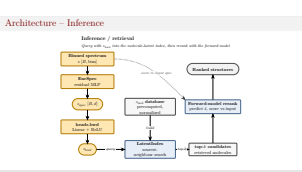
Phase A trains the forward direction, molecule to spectrum. The molecule and fragment latents are averaged, passed through the forward head, and used to condition the decoder.

The decoder is the part worth highlighting. It does not emit a free-form spectrum. It predicts one intensity per fragment and scatters it onto that fragment's m/z bin — which is why the per-fragment embeddings and masses come in from the right. The result is masked by the parent mass, then scored with cosine plus Wasserstein: the dotted line down the left margin.

Phase B trains the backward direction, spectrum to molecule. InfoNCE pulls each spectrum latent onto its own molecule and away from the others — and this is where the molecule encoder is actually trained. The same decoder must then rebuild the spectrum from that backward latent.

Architecture – Inference

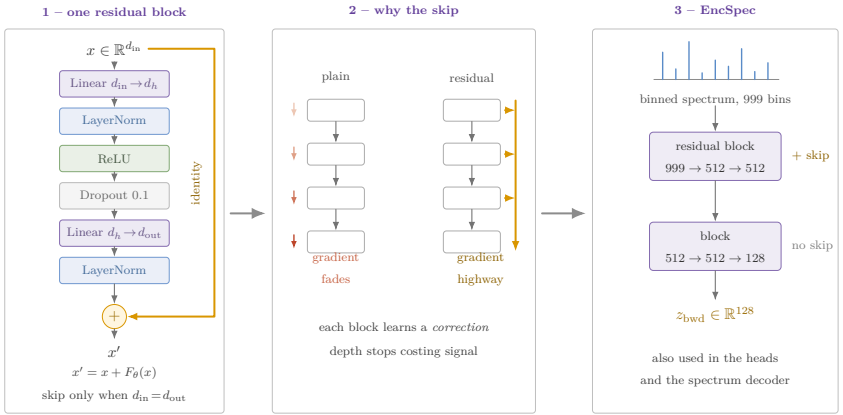




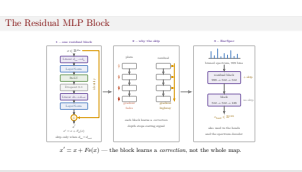
We have a spectrum and want the structure, so nothing on the molecule side is available as input any more. The spectrum takes the same backward path as in training — binned spectrum, residual MLP, backward head — and gives one vector in the shared latent space.
The molecule latents for the whole candidate library are precomputed and normalized in advance. A query is then a nearest-neighbour lookup that returns the top-k molecules.
The second stage is the rerank: for each candidate we run the forward model, predict its spectrum, and score it against the spectrum we measured — the dotted line back to the input. Imagine it like this: Retrieval proposes, the forward model checks.

The rerank also carries a parent-mass mask, and that mask turns out to do most of the work.

The Residual MLP Block



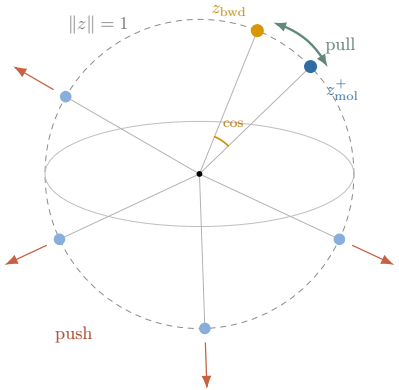
$x' = x + F_\theta(x)$ — the block learns a *correction*, not the whole map.

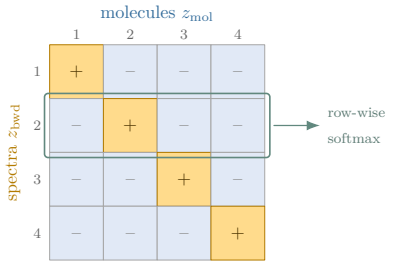


The block that shows up everywhere in the architecture as “residual MLP”: the spectrum encoder, the forward and backward heads, and the spectrum decoder are all built from it.
Unrolled, on the left: a linear layer, LayerNorm, ReLU, dropout, a second linear layer, another LayerNorm — and then the input is added back on. The skip is only taken when the input and output widths agree; where the block changes width, it is a plain MLP.
Why bother, in the middle. Without the skip, the signal — and on the way back the gradient — has to pass through every weight matrix in the stack, and it attenuates. With it there is an unobstructed path from the loss to every layer. The consequence for what the layer has to learn is the more useful way to see it: the identity is already there for free, so each block only has to represent the deviation from it. Small weights mean the block is close to a no-op rather than close to destroying its input, which is why deeper stacks stay trainable.
Concretely on the right, the spectrum encoder: the binned spectrum, 999 bins over one to a thousand Dalton, goes into a residual block of width 512, then a second block projects to the 128-dimensional latent. Only the first block is residual — the second changes width, so there is nothing to add.

Normalization inside is LayerNorm rather than BatchNorm throughout, because several of these blocks are applied per fragment, where the leading dimension is a variable fragment count rather than a batch.

Phase B – InfoNCE Alignment



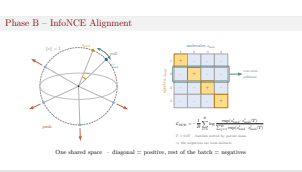


$$\mathcal{L}_{\text{NCE}} = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(z^i_{\text{bwd}}\cdot z^i_{\text{mol}}/T)}{\sum_{j=1}^{B}\exp(z^i_{\text{bwd}}\cdot z^j_{\text{mol}}/T)}$$

$T = 0.07$ batches sorted by parent mass

$\Rightarrow$ the negatives are near-isobaric

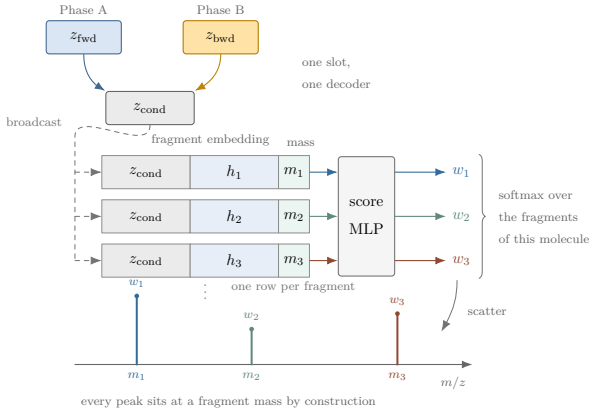
One shared space · diagonal = positive, rest of the batch = negatives



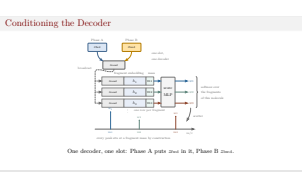
A closer look at the alignment term, because it is what makes retrieval possible at all.
Spectrum and molecule are encoded by completely different networks, so nothing forces their outputs into the same space. InfoNCE does. Both latents are L2-normalised — left panel — so only the direction carries information, and the similarity is a cosine. The loss pulls a spectrum onto the molecule it was actually measured from, and pushes it away from every other molecule in the batch.
Concretely — right panel: within a batch of size B I form the full $B \times B$ matrix of cosine similarities, divide by a temperature of 0.07, and read each row as a B-way classification problem whose correct answer is the diagonal. The loss is the row-wise cross-entropy; the other members of the batch are the negatives, so no sampling machinery is needed.
Two things are worth stressing. First, this is the term that actually trains the molecule encoder — in Phase B the gradient reaches it through nothing else. Second, the batches are not random: they are sorted by parent mass and cut into contiguous chunks, so the negatives a spectrum must reject are its mass-nearest neighbours. Mass can no longer separate the positive from the negatives — exactly the confound the results section is about.

And this is the space the retrieval lookup runs in: the candidate library is encoded once, and a query spectrum is a cosine search.

Conditioning the Decoder



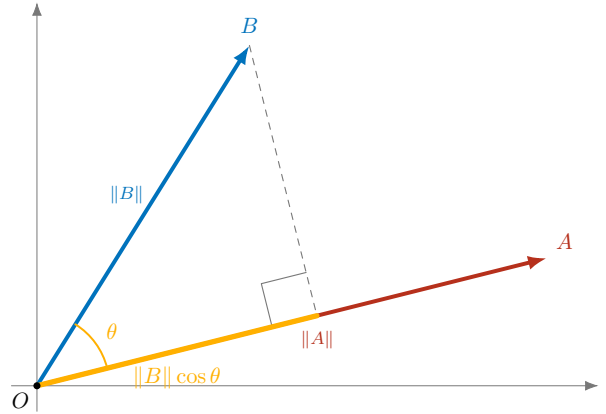
One decoder, one slot: Phase A puts z_{fwd} in it, Phase B z_{bwd}.



What “conditioning” means concretely in the architecture diagram.
The decoder never sees a molecule. It sees the fragment set, and one global vector — z_{cond}. That vector is the only opening through which anything about the input reaches it.
Mechanically: the latent is broadcast, unchanged, to every fragment of the molecule and concatenated onto that fragment’s own embedding and its mass. One shared MLP scores each row, and a softmax over the fragments turns the scores into intensity shares. Each share is then added into the m/z bin given by that fragment’s mass — so the peaks are placed by chemistry, and the conditioning latent only decides how the intensity is distributed over them. This is also why the decoder cannot collapse to the dataset-mean spectrum: its output is a function of the actual fragment set.

The point of the slide is the slot itself. In Phase A the forward head’s output goes in, in Phase B the spectrum’s backward latent goes in — same weights, same decoder, no switch inside. So the same reconstruction loss grades both latents through one shared function, and a z_{bwd} that does not sit where z_{fwd} sits cannot score well. The shared decoder pushes the two directions into one space; InfoNCE does the rest.

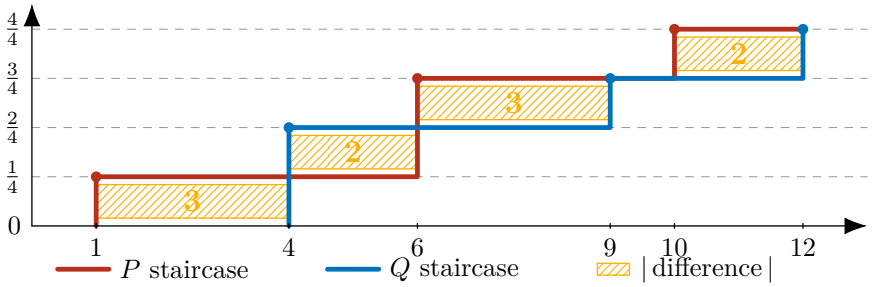
Cosine Similarity – Geometry



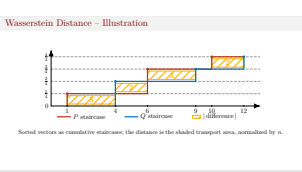
$\cos\theta = (A \cdot B)/(\|A\|\|B\|)$ — direction only, hence scale-invariant.

Cosine similarity is the angle between the two intensity vectors. It depends on direction alone, so rescaling a spectrum leaves it unchanged.

Wasserstein Distance – Illustration

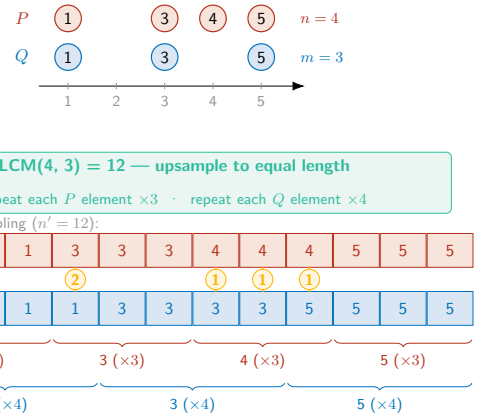


Sorted vectors as cumulative staircases; the distance is the shaded transport area, normalized by n.

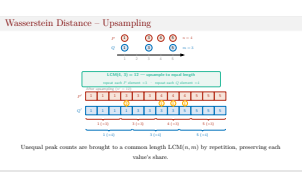


The Wasserstein distance as a transport cost: both sorted vectors are drawn as cumulative staircases, and the shaded area between them — divided by the number of elements — is the distance.

Wasserstein Distance – Upsampling



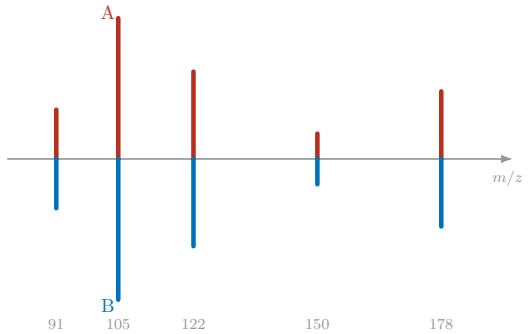
Unequal peak counts are brought to a common length LCM(n, m) by repetition, preserving each value's share.



The matched-pair form requires both vectors to be of equal length, which real spectra never are. Each is therefore repeated up to the least common multiple, which preserves every value's relative share of the total mass.

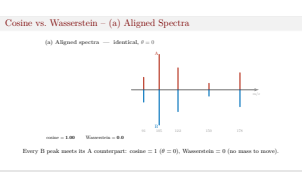
Cosine vs. Wasserstein – (a) Aligned Spectra

(a) Aligned spectra — identical, $\theta = 0$



cosine = **1.00** Wasserstein = **0.0**

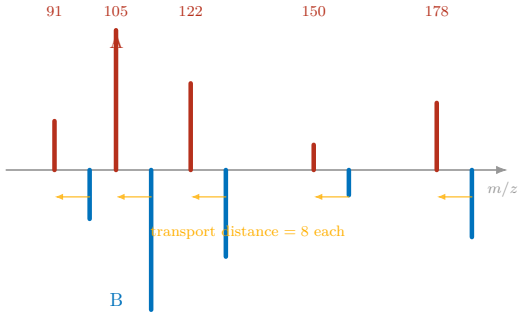
Every B peak meets its A counterpart: cosine = 1 ($\theta = 0$), Wasserstein = 0 (no mass to move).



The reference case: spectrum A plotted upward, spectrum B mirrored downward, peaks aligned. Both measures agree that these are identical.

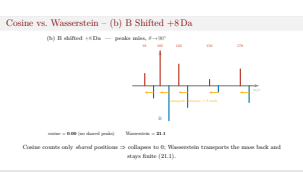
Cosine vs. Wasserstein – (b) B Shifted +8 Da

(b) B shifted +8 Da — peaks miss, $\theta \to 90°$



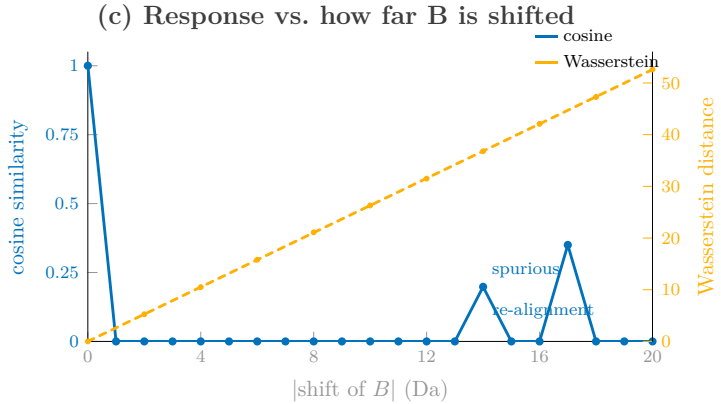
cosine = **0.00** (no shared peaks) Wasserstein = **21.1**

Cosine counts only *shared* positions ⇒ collapses to 0; Wasserstein transports the mass back and stays finite (21.1).

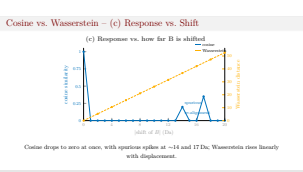


Now shift B by 8 Dalton. Cosine registers intensity only at shared positions, so its dot product vanishes and it drops straight to zero — although the spectra are evidently still related. Wasserstein measures the work required to transport B's mass back onto A, and returns a finite value.

Cosine vs. Wasserstein – (c) Response vs. Shift

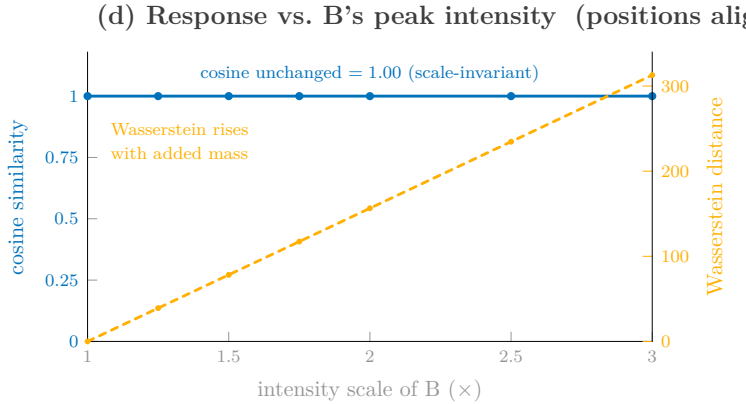


Cosine drops to zero at once, with spurious spikes at ∼14 and 17 Da; Wasserstein rises linearly with displacement.

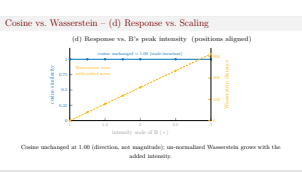


Sweeping the shift makes this systematic. Cosine falls off abruptly and even exhibits spurious spikes, where a displaced peak coincides with a different peak. Wasserstein degrades gracefully, in proportion to the displacement.

Cosine vs. Wasserstein – (d) Response vs. Scaling



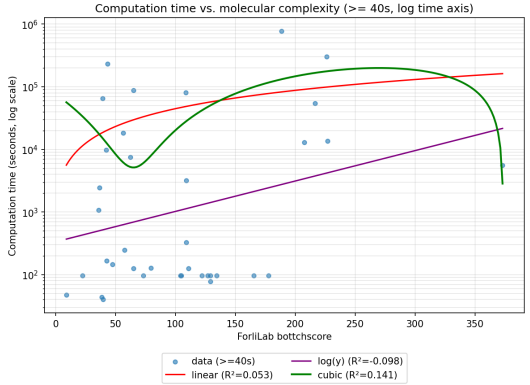
Cosine unchanged at 1.00 (direction, not magnitude); un-normalized Wasserstein grows with the added intensity.



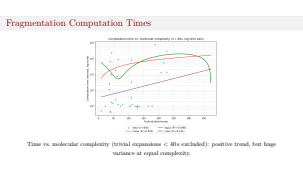
And the complementary axis: B's intensities are scaled with the positions held fixed. Cosine is unaffected, as it compares direction, not magnitude, whereas the un-normalized Wasserstein distance grows with the added mass.

The two measures therefore capture different properties — pattern at fixed coordinates versus position along the axis — which is why the loss employs both.

Fragmentation Computation Times

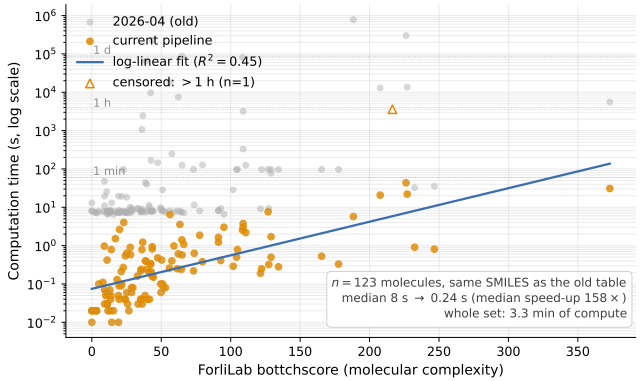


Time vs. molecular complexity (trivial expansions $< 40\,\mathrm{s}$ excluded): positive trend, but huge variance at equal complexity.

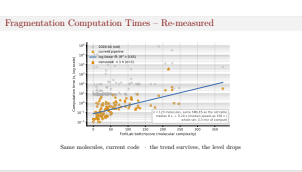


Cost against molecular complexity, on a logarithmic scale, with the trivial expansions excluded. The trend is clearly positive, but the spread at any given complexity is very large — complexity alone therefore does not predict runtime; the available fragmentation pathways matter just as much.

Fragmentation Computation Times – Re-measured



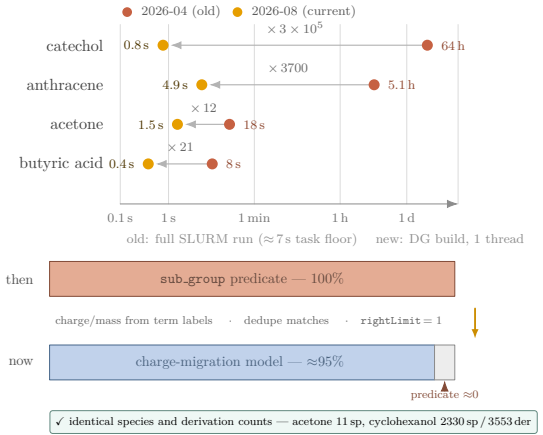
Same molecules, current code · the trend survives, the level drops



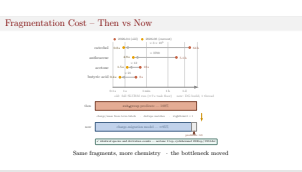
The same experiment as the previous slide, re-run with today's pipeline — identical molecule list, identical SMILES, identical complexity axis.
Grey is the old measurement, amber the current one. The whole cloud drops by two to four orders of magnitude: the median molecule goes from eight seconds to a quarter of a second, and all hundred-odd molecules together now cost three and a half minutes of compute rather than the better part of a fortnight. What does not change is the shape. The trend with complexity is still positive, and the spread at fixed complexity is still wide — the fit explains under half the variance. So the conclusion from the old slide stands: complexity alone does not predict runtime, the available fragmentation pathways do. Only the level moved.

On comparability: the old numbers are full cluster runs, which is why they pile up on a floor around seven to ten seconds — that floor is container start-up, not chemistry. The new numbers are the DG build alone, one thread, through the same strategy `main.py` uses. And the tail has not vanished: the open triangle is scopolamine, which did not finish inside the one-hour limit I gave each task, so it is censored rather than dropped. One further molecule is absent because its SMILES in the old table does not parse.

Fragmentation Cost – Then vs Now



Same fragments, more chemistry · the bottleneck moved



The previous slide is the old world — those timings were measured in April. This is where the same pipeline stands today.

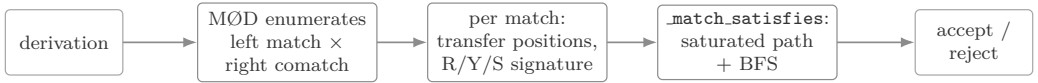
Upper panel: catechol went from sixty-four hours to under a second, anthracene from five hours to five seconds. Small molecules move much less, because they are dominated by start-up rather than by fragmentation. The comparison is not perfectly controlled — old numbers are full cluster runs with a seven-second task floor, new ones in-process builds — but three to five orders of magnitude are not a measurement artefact.

Lower panel, why. In June the `sub_group` predicate was the entire per-molecule runtime. Three commits landed on it: charge and mass are read off the term labels, matches are deduplicated by a generalized-position signature, and the vertex mapper is capped at one right-side comatch, so MØD no longer enumerates every mapping to service a Python callback.

Two points. This is optimisation, not skipped work: acetone returns the same eleven species it returned at two minutes. And the cost that remains is the delocalized charge-migration model, which did not exist in the old measurement — the pipeline is orders of magnitude faster while doing strictly more chemistry.

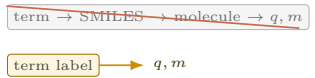
Fragmentation Cost – What Moved

The hot path, 2026



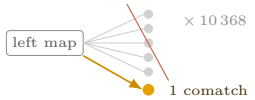
What moved it

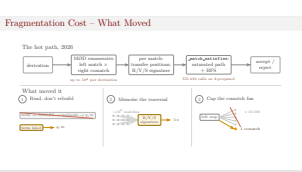
1. Read, don't rebuild



2. Memoise the traversal

3. Cap the comatch fan

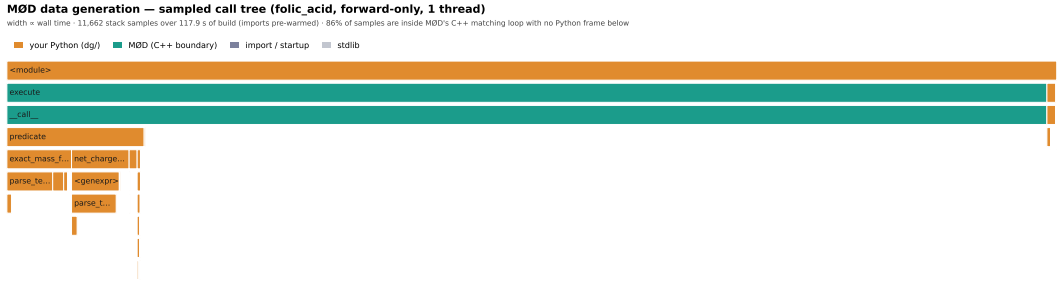




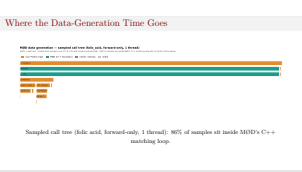
The previous slide says the cost fell by three to five orders of magnitude. This is where it went.
Top row is the June hot path. For every derivation, MØD enumerated the full product of left matches and right comatches — up to ten thousand on a symmetric molecule — and every one of them called back into Python and ran the saturated path and BFS traversal: half a million traversals on 2-propanol. That whole stretch was a hundred percent of the runtime.
Three commits, in the middle row. First: charge and mass are read off the term vertex labels instead of rebuilding a molecule per fragment. Second: the verdict only depends on where the rule's alkyl, heteroatom and saturated positions land, so the traversal is memoised on that signature and symmetry-equivalent embeddings collapse — half a million calls down to four hundred and twenty. Third, and the largest: the outcome depends on the left map alone, so right-side comatch multiplicity is pure redundancy and the mapper is capped at one. 2-propanol goes from twenty seconds to one and a half.

The green band is the part I want to insist on, because a filter that got cheap by filtering less would be cheap for the wrong reason. The dumps are byte-identical. Cyclohexanol gives the same species and derivation counts capped or uncapped — and is one point seven times slower uncapped, for exactly the same graph. The predicate still rejects fifteen point six percent of its species, and acetone returns the same eleven species it returned at two minutes.

Where the Data-Generation Time Goes



Sampled call tree (folic acid, forward-only, 1 thread): 86% of samples sit inside MØD's C++ matching loop.



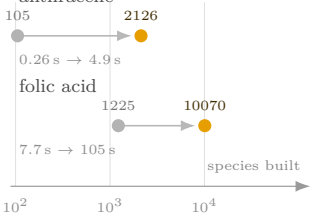
And a profile of where that time is actually spent. The overwhelming majority of samples lie within MØD's C++ matching loop, with no Python frame below — the Python-side predicates are therefore not the bottleneck, and accelerating this is not a matter of optimizing my own code.

What the Speedup Bought

① New chemistry

charge migration: off → on

anthracene

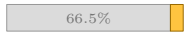


95% of runtime today — and it buys glucose p=3.7×10^{-5}.

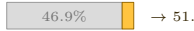
② The hard tail

molecules that used to time out

progesterone 124 → 148 masses



cholesterol 164 → 188 masses

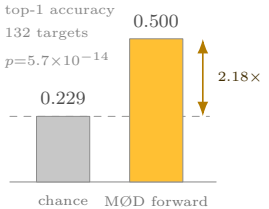


% of NIST intensity explained

✓ aldosterone: killed at 12 h30 → 1 h14. A timeout writes no dump.

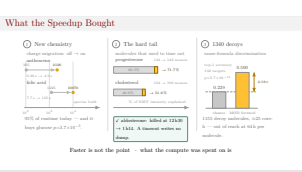
③ 1340 decoys

same-formula discrimination



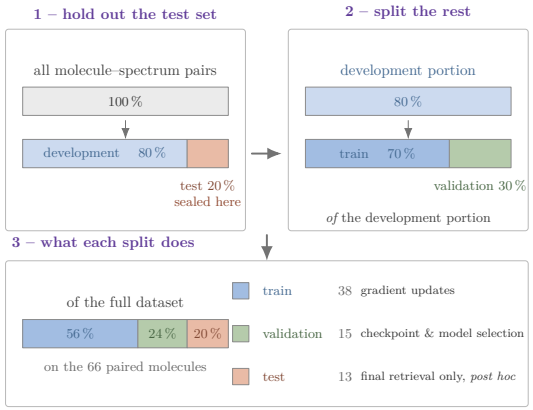
1155 decoy molecules, ≈25 core-h — out of reach at 64 h per molecule.

Faster is not the point · what the compute was spent on is



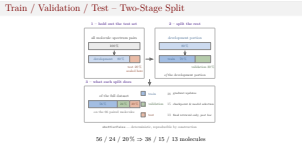
The previous two slides are about cost. This one is about what the cost bought. Three things.
First, chemistry I could not previously afford. The delocalized charge-migration model is now about ninety-five percent of the runtime, and it multiplies the species count by twenty on anthracene. In the old regime, where a single predicate cost two minutes on acetone, it could not have been switched on at all. It is not cosmetic either: on glucose, limonene and sucrose the gains survive chance correction.
Second, the hard molecules stopped disappearing. One that exceeds its wall limit writes no dump and silently drops out of the corpus — which biases the data against exactly the difficult cases. Aldosterone was killed at twelve and a half hours; capped it finishes in an hour and a quarter, and on the two steroids where the cap was validated the result is a strict superset, twenty-four extra masses each.
Third, scale. The Phase-1 gate needed fragmentation graphs for one thousand three hundred and forty same-formula decoys, and it is what says the forward direction carries structural information at all: top-one of one half against a chance level of point two-three, p of five times ten to the minus fourteen. At sixty-four hours for catechol alone, that experiment would not have been attempted.

Train / Validation / Test – Two-Stage Split



`shuffle=False` — deterministic, reproducible by construction

56 / 24 / 20 % ⇒ 38 / 15 / 13 molecules



How the data are partitioned, since every reported number depends on it.
The split is made in two stages. First the full set of molecule–spectrum pairs is cut eighty–twenty, and that twenty percent is the test set — it is sealed at this point and is not looked at again until the final evaluation. Then the remaining development portion is cut seventy–thirty into training and validation. Note that the second ratio is relative to the development portion, not to the whole dataset, so what comes out overall is fifty-six, twenty-four and twenty percent.
On the paired subset — the sixty-six molecules that have both a fragmentation graph and a measured spectrum — that is thirty-eight, fifteen and thirteen.
The roles are strictly separated, and this is the point of the slide. Training data drive the gradient updates. The validation split does model selection: which checkpoint is kept, and the hyperparameters. The test split does neither; it is used post hoc for the retrieval evaluation only. So nothing from the evaluation data can leak back into the fit.
One detail worth naming: the partition is made with `shuffle=False`, so the data are cut in their original order rather than permuted first. The split is therefore deterministic and reproducible by construction — the random state has no effect in this configuration.

The obvious caveat: with thirteen test molecules, the test estimate is noisy. That is a consequence of how few molecules have both a derivation dump and a measured spectrum, and it is part of why I call this a proof of concept.

Splits, Gallery and Provenance

Molecules with paired dumps + spectra	66
train / validation / test	38 / 15 / 13
Retrieval gallery (train + val + test)	66
Mass-controlled queries (test)	13
Training runs behind the numbers	1
Spectrum bins (1–1000 Da, 1 Da)	999

All reported numbers come from one training run (`ml_checkpoint.pt`, 2026-07-06) — no seed averaging, no model selection on the test split.

Splits, Gallery and Provenance

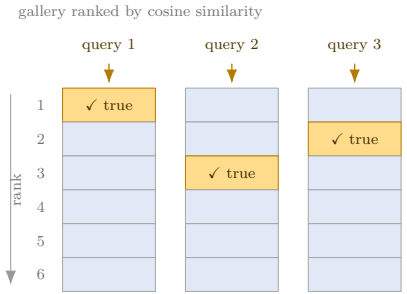
Molecules with paired dumps + spectra	66
train / validation / test	38 / 15 / 13
Retrieval gallery (train + val + test)	66
Mass-controlled queries (test)	13
Training runs behind the numbers	1
Spectrum bins (1–1000 Da, 1 Da)	999

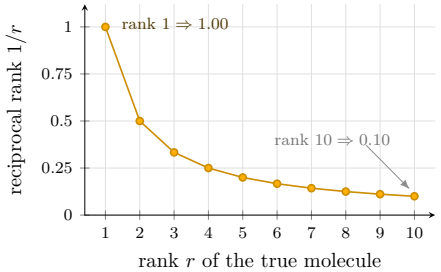
All reported numbers come from one training run (ml_checkpoint.pt, 2026-07-06) — no seed averaging, no model selection on the test split.

The figures underlying the results slides. 66 molecules had both a fragmentation graph and a measured spectrum, which splits 38 / 15 / 13. Retrieval is closed-library: the gallery comprises all 66, which is the structure-elucidation setting, and the 13 test molecules were seen neither in training nor in model selection.

All results come from a single training run, without seed averaging. That is the principal reason I describe the work as a proof of concept.

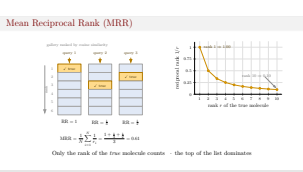
Mean Reciprocal Rank (MRR)





$$\mathrm{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{r_i} = \frac{1+\frac{1}{3}+\frac{1}{2}}{3} = 0.61$$

Only the rank of the *true* molecule counts · the top of the list dominates



The retrieval metric, since every result slide is stated in it.
For one query I rank the whole gallery by cosine similarity to the query spectrum and look up where the true molecule landed. The score for that query is one over its rank: rank one gives 1, rank three gives a third. MRR is that reciprocal rank averaged over the queries — left panel.
Two properties matter for reading the results. It only cares about the position of the correct answer; everything above it is wrong by construction and nothing below it is looked at. And the weighting is strongly top-heavy — right panel: moving a hit from rank ten to rank one is worth ten times as much as moving it from rank twenty to rank ten. That matches how retrieval is actually used — a chemist inspects the first few candidates, not the first hundred.

Two cautions. With a gallery of only 66 molecules an MRR is easy to over-read, which is why I always show it against a chance level rather than on its own — that is the next slide. And a single bad query moves the average much less than a single good one, because the score is bounded above by one and floors near zero; MRR is therefore optimistic in the tail.

Analytic Chance Level in a Mass Window

A query restricted to $\pm W$ Da leaves a pool of m candidates, exactly one of which is correct. Ranking that pool at random puts the correct molecule at rank r with probability $1/m$, so

$$\mathrm{MRR}_{\mathrm{chance}}(m) = \mathbb{E}\left[\tfrac{1}{r}\right] = \frac{1}{m}\sum_{r=1}^{m}\frac{1}{r} = \frac{H_m}{m}, \quad \text{chance} = \frac{1}{N}\sum_{i=1}^{N}\frac{H_{m_i}}{m_i}$$

H_m: the m-th harmonic number; m_i: pool size of query i.

m	1	2	3	5	10	20
H_m/m	1.000	0.750	0.611	0.457	0.293	0.180

The average is taken *per query*, not at the mean pool size: H_m/m is convex, so plugging in $\bar{m} = 5.4$ would give $\approx$0.44 instead of 0.503 — the per-query bar is the **harder** one.

Analytic Chance Level in a Mass Window

A query restricted to $\pm W$ Da leaves a pool of m candidates, exactly one of which is correct. Ranking that pool at random puts the correct molecule at rank r with probability $1/m$, so

$$\mathrm{MRR}_{\mathrm{chance}}(m) = \mathbb{E}\left[\tfrac{1}{r}\right] = \frac{1}{m}\sum_{r=1}^{m}\frac{1}{r} = \frac{H_m}{m}, \quad \text{chance} = \frac{1}{N}\sum_{i=1}^{N}\frac{H_{m_i}}{m_i}$$

H_m: the m-th harmonic number; m_i: pool size of query i.

m	1	2	3	5	10	20
H_m/m	1.000	0.750	0.611	0.457	0.293	0.180

The average is taken *per query*, not at the mean pool size: H_m/m is convex, so plugging in $\bar{m} = 5.4$ would give $\approx$0.44 instead of 0.503 — the per-query bar is the **harder** one.

Where the chance numbers on the mass-controlled slide come from.
Inside a window the query has a pool of m candidates, and exactly one of them is right. If I rank that pool at random, the correct molecule is equally likely to land at any rank, so each rank has probability one over m. The expected reciprocal rank is then the average of one over r over all m ranks — that is the m-th harmonic number divided by m.
Every query has its own pool size, so I evaluate that expression per query and average afterwards. Small pools are easy: with three candidates, chance alone gives 0.61.

One detail worth stating. Because the function is convex, averaging per query gives a higher value than plugging in the mean pool size — 0.503 rather than about 0.44. So the bar I report is the stricter of the two, and the margin I quote is the conservative one.

Honest assessment

- Beyond-mass margin is *small*: +0.048 / +0.013 MRR on 13 queries, single run
- Uncontrolled metrics — reranking included — are mass filtering, not evidence
- Fragment branch buys molecule-specificity, not a retrieval win

Now the honest assessment. What survives the control is small. Roughly five and one percentage points of MRR, on thirteen queries and a single run. The uncontrolled metrics, reranking included, reflect mass consistency. I do not present them as evidence. The fragment branch earns its place through molecule-specificity, not through a retrieval gain. Which leads directly to the outlook.

Training Hyperparameters

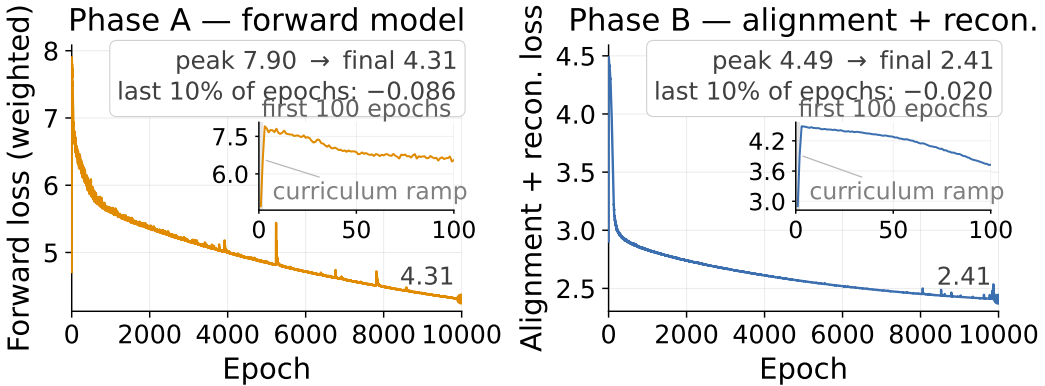
Optimizer	Adam (one per phase)
Learning rate / weight decay	7.1×10^{-5} / 1.4×10^{-4}
Batch size	64
Epochs (Phase A + Phase B)	$10\,000 + 10\,000$
Shared latent dimension d	256
GINEConv layers / width	3 / 128
Fragmentation-graph GCN	2× GCNConv, mean pool
Dropout / normalization	0.43 / BatchNorm
Cosine / Wasserstein weight	1.59 / 1.17
Cycle (fwd / spec) weight	0.77 / 0.089
Align weight / temperature τ	0.30 / 0.07
Curriculum start / increment	0.73 / 0.17

Tuned entries: best of a 20-trial Optuna sweep; checkpoint selection = final epoch.

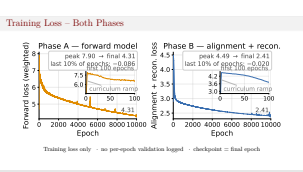
Training Hyperparameters
Optimizer Adam (one per phase)
Learning rate / weight decay 7.1×10^{-5} / 1.4×10^{-4}
Batch size 64
Epochs (Phase A + Phase B) $10\,000 + 10\,000$
Shared latent dimension d 256
GINEConv layers / width 3 / 128
Fragmentation-graph GCN 2× GCNConv, mean pool
Dropout / normalization 0.43 / BatchNorm
Cosine / Wasserstein weight 1.59 / 1.17
Cycle (fwd / spec) weight 0.77 / 0.089
Align weight / temperature τ 0.30 / 0.07
Curriculum start / increment 0.73 / 0.17
Tuned entries: best of a 20-trial Optuna sweep; checkpoint selection = final epoch.

And the hyperparameters, for reference. The tuned entries are the best trial of a twenty-trial Optuna search — learning rate, weight decay, batch size, latent dimension, dropout, normalization, the loss weights and the curriculum. The remainder were fixed a priori.

Training Loss – Both Phases



Training loss only · no per-epoch validation logged · checkpoint = final epoch



The training curve, since the epoch count on the previous slide usually prompts the question.
Two panels, because the two phases optimize different objectives — the forward loss and the alignment-plus-reconstruction loss are not comparable numbers, and I do not want them read as one curve. Phase A falls from 7.9 to 4.3, Phase B from 4.5 to 2.4. Ten thousand epochs each, about six hours on one GPU.
The rise at the very start is not instability, it is the curriculum. The hard-negative fraction is 0.73 in the first epoch and reaches one by the third — the two numbers on the hyperparameter slide. The task gets harder while the model is learning it, so the loss climbs before it falls. That is what the inset shows.
The occasional one-epoch spikes in Phase A are the hard-negative loader being rebuilt every epoch: a batch composition that happens to be hard, and gone again by the next epoch.
Two things I would rather state than be asked. First, both curves are still descending at the end — the last tenth of the run is worth another 0.09 and 0.02 — so training ended on budget, not on convergence, and the checkpoint is simply the final epoch. There is no early stopping and no model selection here.

Second, and this is the real limitation: it is training loss only. I did not log validation per epoch, so this figure cannot tell you whether the model overfits, and I do not claim that it does not. What speaks to generalization is the mass-controlled result on the held-out test split, not this slide.

Model Size – Where the Parameters Are

Module	Parameters	Share
Molecule encoder EncMol (3× GINEConv, w=128)	168 320	9.9%
Fragmentation-graph GCN (2× GCNConv)	33 152	1.9%
Spectrum encoder EncSpec (999→512→512→256)	1 170 688	68.8%
Fragment-grounded decoder DecSpecFragment	198 657	11.7%
Task heads (fwd / bwd + loss $\log \sigma^2$)	131 586	7.7%
Total (trainable)	**1 702 403**	

The single 999×512 input matrix of the spectrum encoder is 30% of the whole model.

The size of the thing, since it is usually the next question. About one point seven million trainable parameters — small by current standards, and deliberately so: the training set is a few thousand molecule–spectrum pairs, so there is no room for a large model.

The distribution is lopsided. Roughly seventy percent sits in the spectrum encoder, and that is almost entirely the first weight matrix: nine hundred ninety-nine bins into five hundred twelve units is half a million parameters on its own, thirty percent of the model, before any learning happens. It is the price of treating the spectrum as a dense vector over m/z.

The graph side — the molecule encoder and the fragmentation-graph convolutions together — is under twelve percent. The part of the architecture that carries the structural claim is the cheap part; the expensive part is the spectrum interface.

Counts are for the checkpoint on the previous slide: latent dimension two hundred fifty-six, BatchNorm.

BatchNorm running statistics are buffers, not parameters, and are not counted here.